

Laboratorio 13: Fragmentación Dinámica, Asignación y Consultas Distribuidas

Contexto

El Ministerio de Salud ha lanzado una iniciativa nacional para recolectar datos de atenciones médicas provenientes de distintos centros de salud en todo el país. El objetivo es analizar el estado de salud de la población y fortalecer la toma de decisiones en políticas públicas.

Para este fin, se ha creado una tabla central llamada `AtencionMedica` en un servidor principal, inicialmente poblada con datos provenientes de hospitales ubicados en Lima y Callao. El siguiente paso consiste en expandir esta iniciativa a nivel nacional.

Se ha optado por fragmentar horizontalmente la tabla `AtencionMedica` según el atributo `Diagnóstico`. Inicialmente se manejarán solo cuatro fragmentos, pero el sistema debe estar preparado para crear nuevos fragmentos de **manera dinámica** conforme se incorporen más diagnósticos y registros. Además, el ministerio ha adquirido varios servidores adicionales para distribuir los datos, mejorando así los tiempos de respuesta para consultas y análisis.

Estructura de la tabla `AtencionMedica` :

```
CREATE TABLE AtencionMedica (  
    DNI CHAR(8),  
    CodMedico INTEGER NOT NULL,  
    Ciudad VARCHAR(50) NOT NULL,  
    Diagnostico VARCHAR(50) NOT NULL,  
    Peso DECIMAL(5,2) NOT NULL,  
    Talla DECIMAL(4,2) NOT NULL,  
    PresionArterial VARCHAR(10) NOT NULL,  
    Edad INTEGER NOT NULL CHECK (Edad >= 0),  
    FechaAtencion DATE NOT NULL  
);
```

Datos de ejemplo:

DNI	CodMedico	Ciudad	Diagnostico	Peso (kg)	Talla (m)	PresionArterial	Edad	FechaAtencio
45781236	101	Lima	Diabetes	70	1.65	130/85	45	2025-01-15
08569321	102	Lima	Hipertensión	85	1.72	150/95	60	2025-01-16
72103654	101	Callao	Obesidad	90	1.60	140/90	35	2025-01-17
25963147	103	Callao	Cardiopatía	78	1.75	145/92	50	2025-01-18
15478962	101	Lima	Diabetes	65	1.58	125/82	42	2025-01-19
36987412	102	Lima	Obesidad	95	1.68	138/88	38	2025-01-20
65412398	103	Lima	Hipertensión	72	1.62	155/98	55	2025-01-21
89632147	101	Callao	Cardiopatía	82	1.70	142/90	48	2025-01-22

Primera Parte: Fragmentación Dinámica y Asignación Distribuida

P1. (1 pts) Creación de particiones

Crear la tabla `AtencionMedica` utilizando particionamiento por el atributo `Diagnóstico`, inicialmente cuatro fragmentos: Diabetes, Obesidad, Cardiopatía, Hipertensión.

Luego, se solicita poblar la tabla con registros sintéticos de atenciones médicas de otras ciudades y con diferentes diagnósticos.

P2. (4 pts) Creación dinámica de nuevos fragmentos

Diseñar un procedimiento almacenado que permita insertar nuevos registros en la tabla `AtencionMedica`. Este procedimiento debe:

- Verificar si ya existe una partición para el diagnóstico de la nueva atención médica.
- Crear dinámicamente una nueva partición si esta no existe.
 - Las particiones se irán generando a medida que se vayan ingresando nuevos datos.
- Insertar `k` registros diferentes que evidencien la creación dinámica del fragmento.

Este enfoque permite escalar el sistema conforme se integren nuevos tipos de diagnósticos en la base de datos.

Nota: Investigar el uso de `triggers` en tablas particionadas.

P3. (5 pts) Asignación distribuida

Utilizar **Docker Desktop** o **Ubuntu Virtualizado** para levantar al menos dos instancias remotas de PostgreSQL. A continuación:

- A partir del procedimiento almacenado anterior, modificar el código para trabajar con fragmentos alojados en otros servidores.
- Distribuir los fragmentos entre un servidor principal (coordinador) y los servidores remotos.
- Usar las extensiones `postgres_fdw` y `dblink` para acceder a tablas foráneas entre servidores.
- Ejecutar **dos consultas** diferentes que evidencien el uso de estas tablas distribuidas.
- Utilizar `EXPLAIN ANALYZE` para analizar el rendimiento de las consultas.

Segunda Parte: Consultas Distribuidas

Esquema de datos

Tabla adicional:

```
CREATE TABLE Pacientes (  
    DNI CHAR(8) PRIMARY KEY,  
    Nombre VARCHAR(50) NOT NULL,  
    Apellidos VARCHAR(100) NOT NULL,  
    FechaNacimiento DATE NOT NULL,  
    Sexo CHAR(1) NOT NULL CHECK (Sexo IN ('M', 'F')),  
    CiudadOrigen VARCHAR(50) NOT NULL  
);
```

Consultas a implementar:

```
SELECT * FROM Pacientes ORDER BY FechaNacimiento;
```

```
SELECT DISTINCT CiudadOrigen FROM Pacientes;
```

```
SELECT Diagnostico, AVG(Edad) AS PromEdad FROM AtencionMedica GROUP BY Diagnostico;
```

```
SELECT * FROM Pacientes NATURAL JOIN AtencionMedica;
```

Configuración inicial:

- Poblar `Pacientes` y `AtencionMedica` con mínimo 50,000 registros sintéticos
- Fragmentar `Pacientes` por rango usando `CiudadOrigen` con vector de partición ["H", "P"]

P4. (4 pts) Algoritmos distribuidos localmente

Diseñar el algoritmo distribuido optimizado para cada consulta asumiendo fragmentos en sitios diferentes, ejecutándose en un único servidor:

- Implementar sentencias SQL del algoritmo distribuido optimizado
- Documentar particiones intermedias en comentarios
- Usar tablas temporales que se eliminen al finalizar la query
- Incluir plan de ejecución con `EXPLAIN ANALYZE` (captura gráfica de pgAdmin)

P5. (6 pts) Algoritmos distribuidos en tres servidores

Implementar los algoritmos distribuidos con arquitectura Master-Worker:

- Utilizar 1 servidor coordinador (master) y 2 servidores remotos (workers)
- Usar `postgres_fdw` o `dblink` para acceso a fragmentos remotos
- Distribuir fragmentos entre los tres servidores
- Ejecutar las cuatro consultas en ambiente distribuido
- Mostrar planes de ejecución evidenciando la integración de servidores
- Scripts SQL completamente replicables

Nota importante: Si P4 no implementa correctamente los algoritmos distribuidos tal como lo indicado en clase, no se asignará puntaje ni para P4 ni P5.

Entregable

Elaborar un documento que contenga:

1. **Explicación de los resultados** obtenidos, incluyendo capturas de pantalla relevantes.
2. **Script SQL** con la estructura y lógica desarrollada. Este script debe ser completamente replicable.

Recursos de apoyo

- [Tutorial postgres_fdw \(Deepnote\)](#)
- [Tutorial DBLINK \(Deepnote\)](#)
- [Referencia técnica en Tencent Cloud sobre FDW](#)
- [PostgreSQL FDW Extension \(documentación oficial\)](#)
- [DBLINK Extension](#)